Hands-On(ish) LLMOps

Maybe Build · Maybe Demo of an Observable AI System in 90 Minutes

---

Participant Packet

Read along. Fill it in. Take it home. Run the workshop again at your own pace whenever you like.

The repo
bit.ly/3ONJiqb → github.com/gigimcc4/DevOpsDays_llmops

The presenter
Jeanne McClure, PhD · Principal · Consultant · Expert
Ars Innovate Technologies & Consulting · www.arsinnovate.com
Postdoctoral Fellow, NC State Data Science & AI Academy

DevOpsDays Raleigh — April 30 / May 1, 2026
© 2026 Jeanne McClure / Ars Innovate. Non-commercial use only. Attribution required.

1 · Get the repo and run setup

Most of you are seeing this repo for the first time right now — that's exactly why the workshop is called Hands-On(ish). If you have Python 3.10+ and Docker already installed, you'll catch up to the live demo within a few minutes. If you don't, switch to demo mode for today and install at home tonight; you still walk out with the working repo, the packet, and everything you need to run it later.

Two things to know about your machine

setup.sh handles Ollama, the models, the Python packages, Langfuse, and Jaeger. The only things it can't install for you are Python and Docker themselves. Check both — fast — before you decide how to participate today:

Python 3.10+

In a terminal, run: `python3.11 --version`

If you see something like Python 3.11.x or 3.12.x, you're set.

If you see "command not found" or 3.9.x — you don't have it ready. Install during/after the workshop:

- Mac (Homebrew, ~2 min): `brew install python@3.11`
- Any OS (official installer, ~3 min): download from https://www.python.org/downloads/

Docker Desktop

In a terminal, run: `docker ps`

If you see a header row with no error, Docker is running. You're set.

If you see "command not found" or "Cannot connect to the Docker daemon" — you don't have it ready. Install during/after the workshop:

- Download: https://www.docker.com/products/docker-desktop (~5–10 min, requires a restart on first run)

Some company laptops block Docker entirely. If yours does and you can't run it: you're a demo-watcher today by default. The packet has everything you need to follow along, and you can install Docker on a personal machine tonight.

Ollama and everything else — setup.sh handles it

If Ollama isn't installed, setup.sh installs it for you. You don't need to do anything ahead of time. (If you're curious, the same install command is: `curl -fsSL https://ollama.com/install.sh | sh`.)

Demo mode is a real option — not a fallback

Roughly half of every workshop room ends up in demo mode their first time. That's expected. You watch the live build on screen, fill in your packet alongside, and run the actual code at home with everything pre-set. By Monday you'll know whether the install gives you trouble in your environment, where you have time and tools to fix it. That's a fine outcome.

Step A — Clone (or download)

```
git clone https://github.com/gigimcc4/DevOpsDays_llmops.git
cd DevOpsDays_llmops
```

No git? Open bit.ly/3ONJiqb in your browser → click the green Code button → Download ZIP → unzip it.

Step B — Open it in your IDE

You'll want your editor open during the workshop so you can read each step's rag.py while we discuss it. Use whatever you already have:

- VS Code: `code .` from inside the folder
- Positron (what Jeanne is using today): File → Open Folder → DevOpsDays_llmops

- Cursor, PyCharm, Sublime, Vim — same idea, point it at the folder

If you've never opened a Python project in an IDE before, today is a perfectly good day to start. Pair up with someone who has.

Step C — Run setup.sh

In your IDE terminal:

```
chmod +x setup.sh
./setup.sh
```

Mac, Linux, or Windows-with-WSL only. Re-running it is safe — every step is idempotent.

In one shot it installs:

- Ollama runtime + the two models we need (~1.5 GB total — the slow part)
- Python packages from requirements.txt
- Langfuse via docker compose (LLM-native observability UI)
- Jaeger via docker compose (OpenTelemetry trace UI)
- Final env-check that confirms everything talks

If setup.sh complains: "Python 3.x is too old (need 3.10+)"

Your Mac is using Apple's old Python (3.9). Install a newer one with Homebrew:

```
brew install python@3.11
```

Then point your terminal at it for this session and re-run setup:

```
mkdir -p ~/bin
ln -sf $(which python3.11) ~/bin/python3
export PATH="$HOME/bin:$PATH"
./setup.sh
```

If setup.sh hangs or you want to run the steps manually

After Ollama is installed, these three commands cover the rest of setup:

```
python3 -m pip install --break-system-packages -r requirements.txt
docker compose up -d
python3 step0_check/check_env.py
```

Step D — Verify the UIs are reachable

Open these in your browser and confirm they load:

- Langfuse: http://localhost:3000 (login: workshop@local.dev / workshop123)
- Jaeger: http://localhost:16686 (no login — empty trace search)

Step E — Get your Langfuse API keys

Step 2 of the workshop reads these keys from your .env file. Without them, the script will exit with "Missing Langfuse keys."

In Langfuse (http://localhost:3000):

1. Log in with workshop@local.dev / workshop123 (created automatically by setup.sh).
2. In the left nav, you should already be inside the "LLMOps Workshop" project. If not, click the project picker at the top.
3. Click the gear icon (Settings) in the bottom-left → API Keys.
4. Click "Create new API key." Copy the Public Key (starts with pk-lf-) and the Secret Key (starts with sk-lf-) — the secret only shows once.

Then in your IDE, rename the example env file to .env and paste your keys into it:

```
mv .env.example .env
```

Open .env (now visible in your IDE's file tree) and fill in your real keys:

```
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_HOST=http://localhost:3000
```

Save the file. Step 2, 3, and 4 of the workshop will now find these keys automatically.

When it succeeds you can open these in your browser:

- Langfuse: http://localhost:3000 (login: workshop@local.dev / workshop123)
- Jaeger: http://localhost:16686

Hiccups → demo mode

If any of these hit you, switch to demo-watcher mode and run it on a personal machine tonight. We can't install Python or Docker for you in 90 minutes.

- Conference wifi too slow for the 1.5 GB model pull
- Corp laptop blocking Docker, pip, or `curl | sh` (Ollama installer)
- Docker Desktop not installed (or daemon not running)
- Windows without WSL
- Port 3000 or 16686 already in use

2 · Get to know our data

RAG is only as good as the documents it retrieves from. Our knowledge base is three short markdown files — printed in full below so you can spot-check what the model retrieves at each step. (After each step you compare the model's answer to what these docs actually say. If they disagree — that's a failure mode, and observability is what tells you which stage failed.)

devops_glossary.md

Observability terms

A trace represents the full journey of a single request through a system. It is composed of one or more spans. In an LLM system, a single user question might produce a trace with spans for: retrieve_context, embed_query, call_llm, format_response.

A span is a named, timed unit of work within a trace. Spans have a start time, an end time, attributes (key-value tags), and events. Parent-child relationships between spans form the trace's shape.

OpenTelemetry (OTel) is the vendor-neutral standard for tracing, metrics, and logs. You instrument your code once using the OTel API, then export to any backend — Jaeger, Tempo, Datadog, Honeycomb — without rewriting instrumentation.

Langfuse is an open-source LLM-specific observability platform. It understands LLM concepts natively: prompts, completions, token counts, model versions, evaluation scores. You can self-host it or use the managed service.

RAG terms

A vector is a fixed-length array of floating-point numbers that represents the semantic content of text. Similar texts produce vectors that are close together in the embedding space.

ChromaDB is an open-source vector database optimized for embeddings. It runs in-process (like SQLite) or as a server, and supports metadata filtering alongside vector search.

Top-k retrieval returns the k chunks whose embedding vectors are nearest to the query vector. Common values are k=3 to k=10 depending on chunk size and context window budget.

Ollama

Ollama is a local runtime for open-weight LLMs. You run `ollama pull <model>` to download weights and `ollama run <model>` to chat. It exposes an OpenAI-compatible HTTP API on port 11434, so libraries that expect OpenAI often work unchanged by pointing at http://localhost:11434.

Small models that fit on laptops include llama3.2:1b (1.3GB), phi3:mini (2.3GB), and qwen2.5:0.5b (400MB). Quality scales with size but so do memory and latency.

llm_observability.md

Traditional APM tools like Datadog, New Relic, and Dynatrace were built for services where a 200 status code means success. LLM-powered services break this assumption. A model can return a syntactically valid response that is factually wrong, semantically off, or politely refuses a reasonable request. The HTTP layer sees success. The user sees garbage.

Three signals unique to LLM systems

Token usage per request is the cost signal. Unlike a database query, every LLM call has variable cost depending on prompt length and output length. You need per-request token counts rolled up by endpoint, user cohort, and feature flag to build a cost model that tracks reality.

Prompt and completion content is the quality signal. Without capturing what went in and what came out, you cannot debug why the model answered the way it did. This requires care: completion data often contains PII, so retention policies need thought. Langfuse and similar tools let you mask fields before they persist.

Retrieval context is the grounding signal. For RAG systems, the retrieved chunks are the ground truth the model is supposed to use. If retrieval brought back the wrong documents, the model's "hallucination" is actually obedient behavior on bad inputs. You cannot diagnose this without logging retrieved chunks alongside the final answer.

When to instrument

Instrument before production. The instinct is to ship first and add observability when something breaks. For LLM systems this is backwards: your first production incident is likely to be a bizarre output with zero context, and you will have no way to reproduce it. Trace from day one.

rag_patterns.md

Retrieval-Augmented Generation grounds an LLM's answer in documents you control. The promise is reduced hallucination and up-to-date answers without retraining. The reality is that RAG pipelines have more moving parts than they appear to, and each part fails in a distinct way.

The five stages

Loading reads raw content from source systems — PDFs, Confluence, Notion, a support corpus. The failure mode here is silent truncation: a library reads 200 of 500 pages and returns no error.

Chunking splits documents into retrievable units. Chunks that are too small strip context and force the model to guess. Chunks that are too large waste tokens and bury the relevant passage. Overlap between chunks helps preserve context across boundaries but inflates the index.

Embedding converts each chunk into a vector. The failure mode is model mismatch: if you embed documents with one model and query embeddings with another, retrieval will be nearly random. This fails silently because both embedding calls succeed.

Retrieval finds the top-k chunks nearest to the query vector. Top-k too low misses relevant passages. Top-k too high dilutes the context window with noise. Distance metric matters: cosine similarity and Euclidean distance rank differently, especially for un-normalized embeddings.

Generation synthesizes an answer from retrieved chunks and the original question. The failure mode at this stage is instruction override: the model answers from its training data instead of from the chunks. Prompting discipline — "answer only using the context below, and if the context is insufficient, say so" — is load-bearing.

Canonical debugging flow

When a RAG answer is wrong, work backwards. First look at the final prompt sent to the model: did retrieval put the right chunks in context? If yes, it is a generation problem — prompt tightening or a stronger model. If no, it is a retrieval problem. Next, look at the retrieved chunks relative to the expected chunks: is the right content in the index at all? If no, it is a loading or chunking problem. If yes but it was not retrieved, it is an embedding or ranking problem.

This flow is impossible without per-stage tracing. This is why observability is not optional for RAG.

3 · Step 1 worksheet — Minimal RAG

This step has no observability. The terminal shows you the answers and that's it. Today's question: what is the absence of signals costing you?

Run the pipeline

```
python step1_rag/rag.py
```

It will ask three canonical questions in sequence. Watch the terminal. Pay attention to what you can — and can't — see.

Per-question table

Fill in your row as the answer prints. Match scale: F = full match (got the key ideas), P = partial, O = off-topic / hallucination / 'I don't have enough information'.

Question	Jeanne's output (live demo)	Your output (if you ran it)	Match? F / P / O	Room poll F:__ P:__ O:__
Q1. Three signals?				
Q2. Why is RAG observability not optional?				
Q3. What does OpenTelemetry give you?				

Group activity (3 minutes — pairs or table)

Looking at Jeanne's printed output for each question:

5. Look at the answer for Q1. Can you tell which source document the model pulled from? Why does that matter?
6. Imagine the answer was wrong. From this terminal output alone, where would your group start the investigation?
7. Write down ONE thing this output isn't telling you that you'd need to know to debug a bad answer:

_____

8. Bonus — if anyone in your group ran it on their laptop: how did your output differ from Jeanne's?

4 · Step 2 worksheet — Add Langfuse

One import, one decorator per stage. Now every question becomes a nested trace at http://localhost:3000.

Run it

python step2_langfuse/rag.py

Open Langfuse in your browser. Click into the most recent trace. Expand the spans.

Per-question table

Now you can see things you couldn't before. Capture them here.

Question	Jeanne's output (live demo)	Your output (if you ran it)	Match? F / P / O	Room poll F:__ P:__ O:__	retrieve latency (ms)	generate_answer latency (ms)	tokens in → out
Q1. Three signals?							
Q2. Why is RAG observability not optional?							
Q3. What does OpenTelemetry give you?							

From any one trace, also note

- Which 3 sources were retrieved? (e.g. devops_glossary.md, rag_patterns.md, llm_observability.md)
- What was the lowest distance score? (Lower = closer match.)
- Click into the generation span. What was the EXACT prompt the model received?

Reflect

Compare your Step 1 row to your Step 2 row for the same question. What can you now answer that you couldn't before?

_____

_____

5 · Step 3 worksheet — OpenTelemetry + Jaeger

Same code pattern, different lens. OTel emits vendor-neutral spans; Jaeger gives you a clickable UI to read them. In production you'd swap Jaeger for Tempo, Datadog, Honeycomb — same instrumentation.

Run it

```
python step3_otel/rag.py
```

Open Jaeger at http://localhost:16686. Search → Service: llmops-workshop-rag → Find Traces.

What you see in the trace tree

Click into the most recent trace. Sketch or list the span names you find, parent → child:

```
ask
 ├─ retrieve
 │   └─ embed        (model = ____________)
 └─ llm.generate     (model = ____________, tokens.in = ____, tokens.out = ____)
```

Per-question observation

Question	Jeanne's output (live demo)	Your output (if you ran it)	Match? F / P / O	Room poll F:__ P:__ O:__	Span tree depth	Total trace duration
Q1. Three signals?						
Q2. Why is RAG observability not optional?						
Q3. What does OpenTelemetry give you?						

Langfuse vs Jaeger — what's each best for?

- Langfuse answers: _____
- Jaeger answers: _____

6 · Step 4 worksheet — Break it. Debug it.

A planted bug. Same code shape, wrong answer, no exception. The traces hold the evidence. Don't peek at the answer key until you've worked through this with your group.

Run the broken version

```
python step4_debug/rag.py
```

The answers will look off — vague, wrong, or 'I don't have enough information.' Don't read the source code. Read the trace.

Trace detective worksheet (pair or group)

Work through these in order. For each one, write what you see — even if you don't yet know what it means.

#	Question to ask the trace	Your observation
1	Open the bad trace in Jaeger. Find the prompt that was sent to the LLM. Does the prompt contain the right context to answer the question?	
2	If no — open the retrieve span. What sources came back? Are they relevant to the question that was asked?	
3	If retrieval is wrong — drill into the embed spans. Look at every embed span's `model` attribute. Are they all the same?	
4	Compare the model used to embed documents (in build_index) vs the model used to embed the query (inside retrieve). Same? Different?	
5	If different: that's your bug. Why does this break retrieval even though both calls succeed?	
6	What's the one-line fix?	

Reflection

This bug was silent. There was no exception. No HTTP 500. The system returned plausible wrong answers. Without observability, how long would this bug have lived in your prod system before someone noticed?

_____

7 · Take this to production

Five swaps to take the laptop demo to a real environment. Your code barely changes.

1. Ollama → hosted model

Replace the Ollama client with OpenAI / Anthropic / Bedrock SDK. The @observe decorators don't change — Langfuse auto-captures both SDKs.

2. Jaeger → your existing APM

Change one env var. The OTLPSpanExporter ships spans wherever you point it: Tempo, Datadog, Honeycomb, New Relic, Grafana Cloud.

3. EphemeralClient → persistent vector store

Chroma HttpClient, Pinecone, Weaviate, pgvector. The retrieve span boundary is identical.

4. Add an evaluation loop

Langfuse LLM-as-judge scores every response against your rubric. Trend the score. Gate releases on it. (This is Phase 2 of the framework on slide 3.)

5. Mask PII before persisting

Langfuse exposes a `mask` hook. Strip emails, account numbers, names from prompts and completions before they reach storage. Plan this with security on day one, not after the first incident.

8 · Resources & Monday checklist

Links

This workshop repo: github.com/gigimcc4/DevOpsDays_llmops
Langfuse docs: langfuse.com/docs
OpenTelemetry Python: opentelemetry.io/docs/languages/python
Jaeger docs: jaegertracing.io
Ollama model library: ollama.com/library
ChromaDB docs: docs.trychroma.com
Ars Innovate: arsinnovate.com

Monday-at-work checklist

Pick one. Just one. Do it next week.

- Wrap one LLM call in your codebase with `@observe()` and point Langfuse at it.
- Add OTLPSpanExporter to your existing service. Pick the LLM call as the first instrumented span.
- Write down your three target signals (cost, quality, grounding) in your team's runbook.
- Identify the one trace your on-call engineer would need to debug a bizarre LLM output. Build the dashboard for that.

Notes

Notes

Work with Jeanne

If today helped your team — and you're wondering what's next — I do this for a living.

Get it right the first time — so you don't pay for a redo

The most expensive part of an AI project isn't hiring help. It's hiring help six months in to clean up a "prototype that became prod by accident." The point of working with me is to skip that detour entirely — and to leave your team owning what we built.

Pick the level of engagement you actually need. Not every project needs all three:

1. Build together. I'm in your repo, design reviews, and decision rooms from day one. Architecture, observability, and evaluation set up correctly the first time — across all three evaluation phases (human eval, LLM-as-judge, production monitoring), not just one slice.

2. Train your team. Engineers pair with me — real PRs, code reviews, runbooks they write themselves. Muscle memory, not slide decks.

3. Optional check-ins. Light-touch follow-ups when you hit something new. Most teams need fewer than they expect — that's the point.

My goal is your team can do this on their own.

Stand up your AI stack — observable from day one

Your team got the "add AI" task. I help you go from blank repo to a working, traced, debuggable system in weeks. Same playbook you saw today, fit to your stack.

Best fit: engineering managers and tech leads who need to ship LLM features without re-living everyone else's first-incident pain.

AI orchestration audits + roadmap reviews

AI already in production and tangled? Inherited a system you can't debug? Drafting the roadmap and want a second set of eyes? I review what's there, name the gaps, and hand you a defensible plan.

Best fit: architects, SREs, and platform teams that have inherited or are scaling AI workloads.

Subcontract / partner

Consulting firm or agency that landed an AI engagement and needs senior depth? I plug in — white-label, fractional, or co-named. You keep the client.

Best fit: independent consultants and boutique agencies expanding their AI services without hiring full-time.

Get in touch

Email: jmcclure@arsinnovate.com
LinkedIn: linkedin.com/in/jeannemcclure
Web: arsinnovate.com

Jeanne McClure, PhD · Principal · Consultant · Expert · Ars Innovate Technologies & Consulting · Postdoctoral Fellow, NC State Data Science & AI Academy

ANSWER KEY

Don't peek until you've worked through Steps 1–4. Note: every model run produces different wording. The 'golden' is the underlying idea you're checking for — not a string match.

Q1 — What are the three signals unique to LLM systems?

Source: llm_observability.md.

1. Cost. Token usage per request — variable cost per call. Without per-request token counts you can't model cost.
2. Quality. Prompt and completion content. Without capturing what went in and what came out, you can't debug why the model answered the way it did.
3. Grounding. Retrieval context. The retrieved chunks ARE the ground truth the model is supposed to use. Wrong chunks → "hallucinations".

Full match (F): all three signals named. Partial (P): one or two named. Off (O): wrong topic, missing all three.

Q2 — Why is RAG observability not optional?

Source: rag_patterns.md (closing line).

Because the canonical RAG debug flow requires per-stage tracing. When an answer is wrong you work backwards: the prompt → the retrieved chunks → the embed model → the index. Without traces, none of those questions can be answered. Debugging is guessing.

F: connects "debugging" or "per-stage tracing" to the answer. P: mentions tracing but no concrete debug flow. O: doesn't mention debug or tracing at all.

Q3 — What does OpenTelemetry give you?

Source: devops_glossary.md.

Vendor-neutral instrumentation. You instrument your code once using the OTel API, then export to any backend — Jaeger, Tempo, Datadog, Honeycomb — without rewriting. It is the standard for cross-service tracing, metrics, and logs.

F: "vendor-neutral" or "instrument once, export anywhere." P: mentions tracing or one backend by name. O: confuses OTel with Langfuse or APMs.

Step 4 — The bug

In retrieve(), the query is embedded with LLM_MODEL (an LLM model — llama3.2:1b) instead of EMBED_MODEL (a dedicated embedding model — all-minilm). Documents were indexed using all-minilm, queries get embedded with llama3.2:1b. Ollama happily returns floats from a chat model, but the vector spaces don't match — so nearest-neighbor search is essentially random.

```
# Broken
qvec = embed_text(query, _model_override=LLM_MODEL)

# Fixed
```

```
qvec = embed_text(query)   # uses EMBED_MODEL via default
```

How the trace reveals it

1. In Jaeger or Langfuse, expand a retrieve → embed span. Read its `model` attribute: llama3.2:1b.
2. Expand any embed span from build_index. Read its `model` attribute: all-minilm.
3. Mismatch. Total time to diagnose with traces: ~30 seconds. Total time to diagnose without traces: indefinite.

Secondary smell

In the retrieve span, hits.min_distance jumps from typical 0.3–0.6 (in step 3) to near 1.0+ (in step 4). When distances spike like that, the embedding space is misaligned — that's a smell worth alerting on in production.

How to use this answer key

Tally your group's matches. Round-by-round in the room:

- "Hands up — who got Full match on Q1?" Count, write on board.
- Repeat for Partial and Off. Same Q2, Q3.
- This makes stochasticity tangible — same code, same prompt, different answers, different match rates.